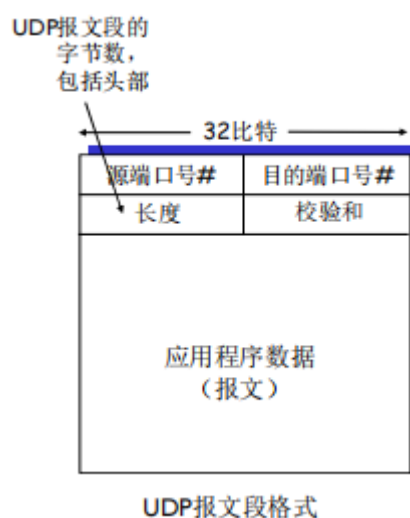


# 第三章 传输层

## 1.UDP

1. best-effort的服务，报文段可能丢失/送到应用进程的报文段乱序
2. 无连接：UDP发送端和接收端之间没有握手；每个UDP报文段都被独立地处理
3. UDP用于：流媒体；DNS；SNMP等
4. 在UDP上可行可靠传输：
  - 在应用层增加可靠性
  - 应用特定的差错恢复
5. UDP数据报：



使用UDP原因：

- 不建立连接（会增加延时）
  - 简单：在发送端和接收端没有连接状态
  - 报文段的头部很小（开销小）
  - 无拥塞控制和流量控制：UDP可以尽可能快的发送报文段
6. UDP校验和：
    - 目标：检测在被传输报文段中的差错（如比特反转）

## 2.面向连接的传输：TCP

### 2.1 TCP概述

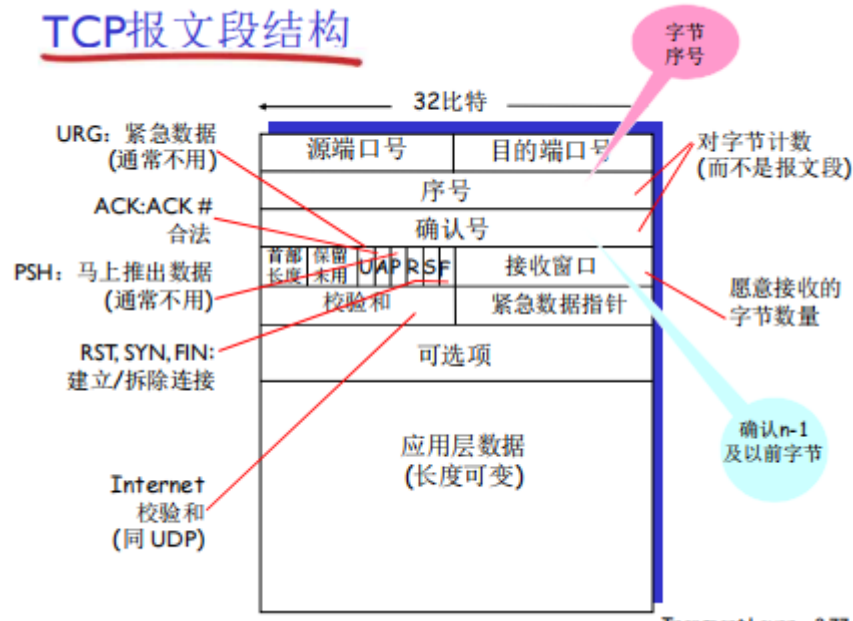
1. 点对点：一个发送，一个接收方
2. 可靠的、按顺序的字节流：没有报文边界
3. 管道化（流水线）：TCP拥塞控制和流量控制设置窗口大小
4. 发送和接收缓存
5. 全双工数据：
  - 在同一连接中数据流双向流动

- MSS: 最大报文段大小

6. 面向连接: 在数据交换之前, 通过握手 (交换控制报文) 初始化发送方、接收方的状态变量

7. 有流量控制: 发送方不会淹没接收方

## 2.2 TCP报文段结构



1. 序号: 报文段首字节的在字节流的编号

2. 确认号:

- 期望从另一个收到的下一个字节的序号
- 累积确认

3. TCP往返延时 (RTT) 和超时:

- 如何设置TCP超时? 比RTT长
  - 太短: 太早超时; 造成不必要的重传
  - 太长: 对报文段丢失反应太慢, 消极
- 怎样估计RTT?
  - SampleRTT: 测量从报文段发出到收到确认的时间  
如果有重传, 忽略此次测量
  - SampleRTT会变化, 因此估计的RTT应该比较平滑  
对几个最近的测量值求平均, 而不是仅用当前的SampleRTT
  - $EstimatedRTT = (1 - \alpha) * EstimatedRTT + \alpha * SampleRTT$   
指数加权移动平均  
过去样本的影响呈指数衰减  
推荐值:  $\alpha = 0.125$
  - 设置超时:  $EstimatedRTT + \text{安全边界时间}$   
 $DevRTT = (1 - \beta) * DevRTT + \beta * |SampleRTT - EstimatedRTT|$   
推荐值:  $\beta = 0.25$
  - 超时时间间隔设置为:

$$TimeoutInterval = EstimatedRTT + 4 * DevRTT$$

## 2.3 可靠数据传输

1. TCP在IP不可靠服务的基础上建立了rdt:

- 管道化的报文段
- 累积确认
- 单个重传定时器
- 是否可以接收乱序的，没有规范

2. 通过以下事件触发重传:

- 超时: 只重发那个最早的未确认段
- 重复的确认: 收到3个冗余确认

3. 产生TCP ACK的建议:

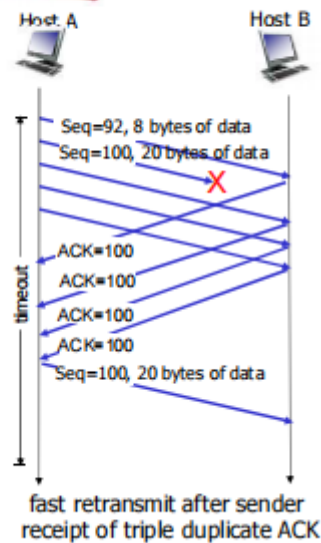
### 产生TCP ACK的建议 [RFC 1122, RFC 2581]

| 接收方的事件                               | TCP接收方动作                                                    |
|--------------------------------------|-------------------------------------------------------------|
| 所期望序号的报文段按序到达。<br>所有在期望序号之前的数据都已经被确认 | 延迟的ACK。对另一个按序报文段的到达最多等待500ms。如果下一个报文段在这个时间间隔内没有到达，则发送一个ACK。 |
| 有期望序号的报文段到达。<br>另一个按序报文段等待发送ACK      | 立即发送单个累积ACK，以确认两个按序报文段。                                     |
| 比期望序号大的报文段乱序到达。<br>检测出数据流中的间隔        | 立即发送重复的ACK，指明下一个期待字节的序号                                     |
| 能部分或完全填充接收数据间隔的报文段到达。                | 若该报文段起始于间隔（gap）的低端，则立即发送ACK。                                |

4. 快速重传:

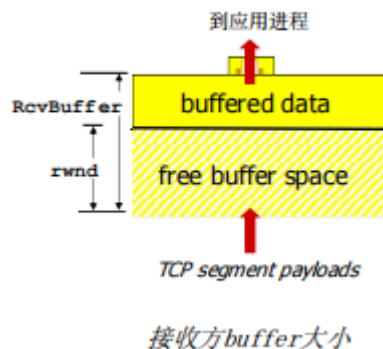
- 超时周期往往太长: 在重传丢失报文段之前的延时太长
- 通过重复的ACK来检测报文段丢失:
  - 发送方通常连续发送大量报文段
  - 如果报文段丢失，通常会引起多个重复的ACK
  - 如果发送方收到同一数据的3个冗余ACK，重传最小序号的段
  - 快速重传: 在定时器过时之前重发报文段

## TCP快速重传

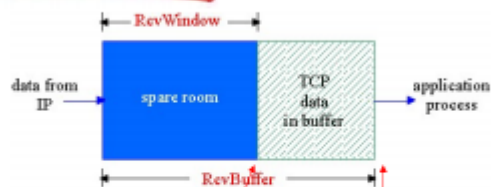


## 2.4 流量控制

1. 接收方控制发送方，不让发送方发送的太多、太快以至于让接收方的缓冲区溢出
2. 接收方在其向发送方的TCP段头部的rwnd字段通告其空闲buffer大小
  - RcvBuffer大小通过socket选项设置
  - 很多操作系统自动调整RcvBuffer
3. 发送方限制未确认字节的个数  $\leq$  接收方发送过来的rwnd值
4. 保证接收方不会被淹没



## TCP流量控制



(假设TCP接收方丢弃乱序的报文段)

□ 缓存中的可用的空间

= RcvWindow

= RcvBuffer - [LastByteRcvd - LastByteRead]

## 2.5 连接管理

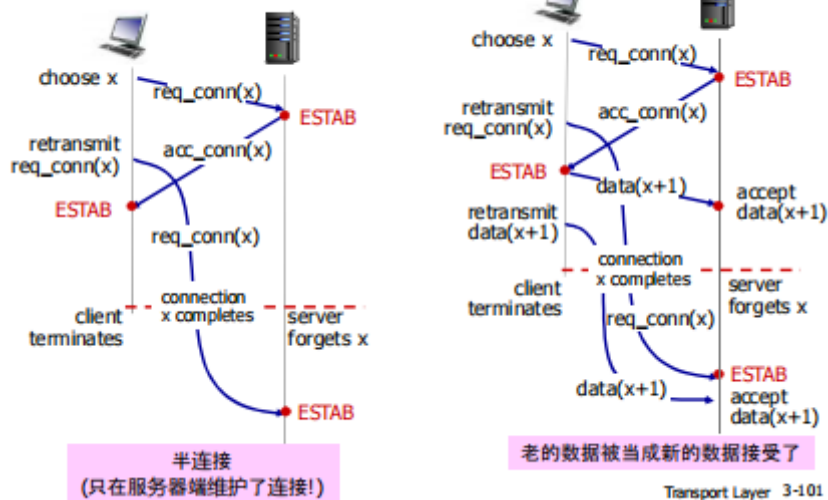
### 1. 同意建立连接：

- 在正式交换数据之前，发送方和接收方握手建立通信关系：
  - 同意建立连接（每一方都知道对方愿意建立连接）
  - 同意连接参数

### 2. 两次握手的问题：

- 请求连接的段没有丢失，但可能超时；由于丢失造成重传；报文乱序；相互看不到对方
- 失败的场景：

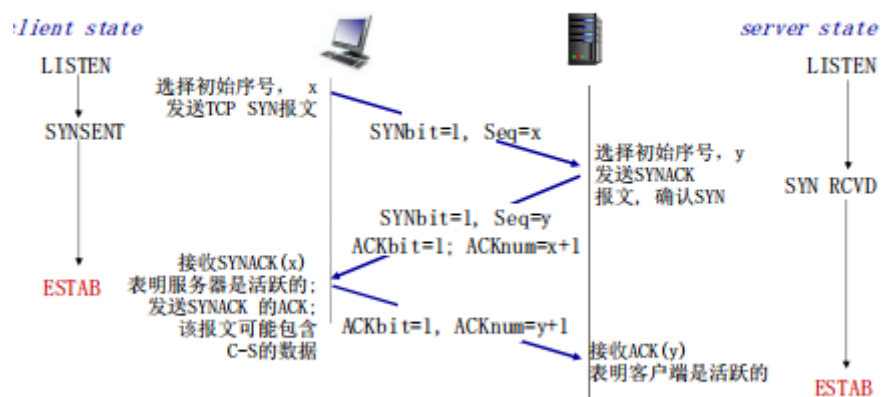
#### 2次握手的失败场景：



- 问题1：半连接
- 问题2：老的数据被当作新的数据接收了

### 3. TCP3次握手：

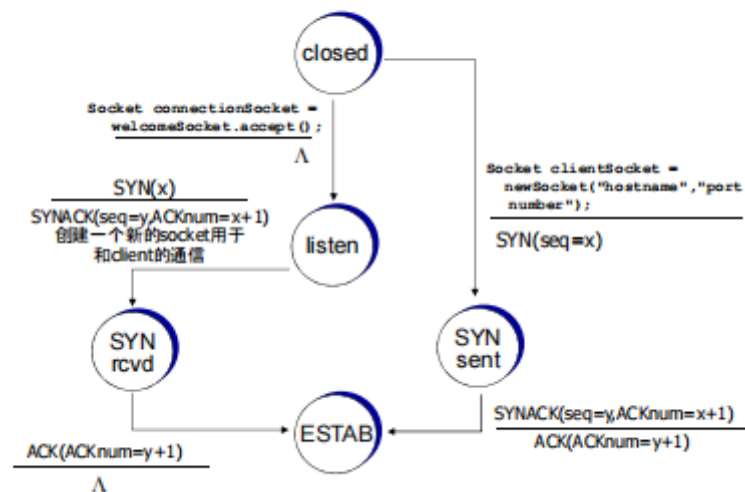
#### TCP 3次握手



解决方案：变化的初始序号+双方确认对方的序号（3次握手）

- 3次握手解决半连接和接收老数据问题

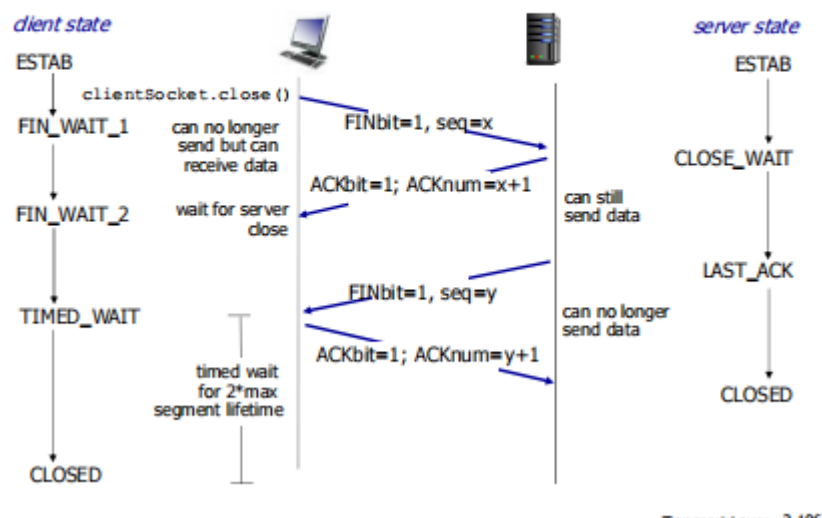
## TCP 3次握手: FSM



### 4. 关闭连接:

- 客户端，服务器分别关闭它自己这一侧的连接  
发送FIN bit=1的TCP段
- 一旦接收到FIN，用ACK回应  
接收到FIN段，ACK可以和它自己发出的FIN段一起发送
- 可以处理同时的FIN交换

## TCP: 关闭连接



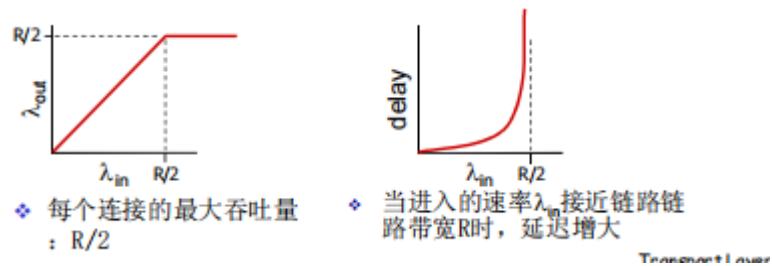
## 3. 拥塞控制原理

### 1. 拥塞:

- 非正式的定义: 太多的数据需要网络传输, 超过了网络的处理能力
- 与流量控制不同
- 拥塞的表现:
  - 分组丢失: 路由器缓冲区溢出
  - 分组经历比较长的延迟: 在路由器的队列中排队

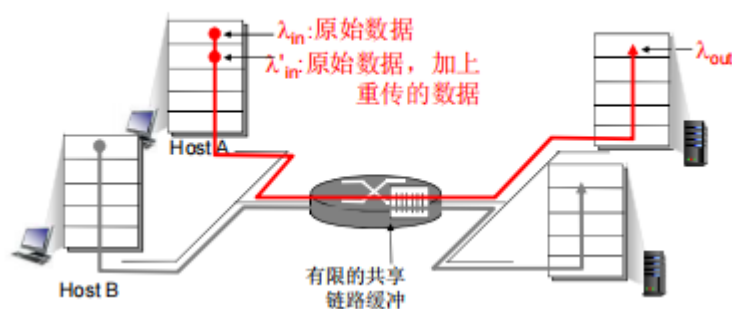
## 2. 拥塞的原因/代价：场景1

- 2个发送端，2个接收端
- 一个router，具备无限大的缓冲
- 输出链路带宽：R
- 没有重传



## 3. 拥塞的原因/代价：场景2

- 一个router，有限的缓冲
- 分组丢失时，发送端重传
  - 应用层的输入=应用层输出： $\lambda_{in} = \lambda_{out}$
  - 传输层的输入包括重传： $\lambda'_{in} \geq \lambda_{in}$



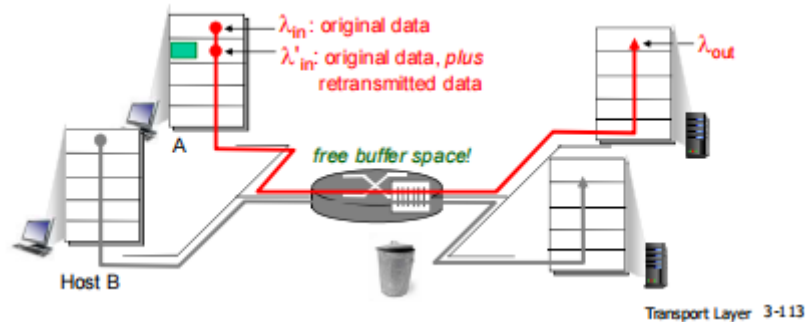
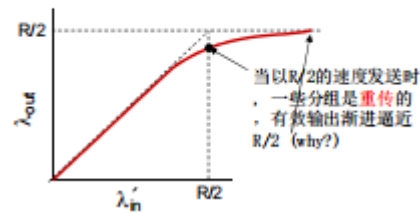
- 理想化：发送端有完美的信息：发送端知道什么时候路由器的缓冲是可用的
  - 只在缓冲可用时发送
  - 不会丢失： $\lambda'_{in} = \lambda_{in}$
- 理想化：掌握丢失的信息：分组可以丢失，在路由器由于缓冲器满而被丢弃
  - 如果发送端知道分组丢失了，发送方重传分组

## 拥塞的原因/代价: 场景2

### 理想化: 掌握丢失信息

分组可以丢失, 在路由器由于缓冲器满而被丢弃

❖ 如果发送端知道分组丢失了, 发送方重传分组



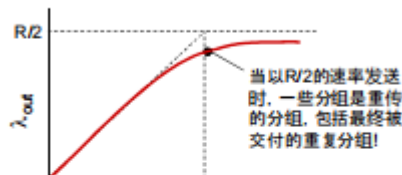
### 现实情况: 重复

- 分组可能丢失, 由于缓冲器满而被丢弃
- 发送端最终超时, 发送第2个拷贝, 2个分组都被传出
- 拥塞的代价:
  - 为了达到一个有效输出, 网络需要做更多的工作 (重传)
  - 没有必要的重传, 链路中包括了多个分组的拷贝
  - 是那些没有丢失, 经历的时间比较长 (拥塞状态) 但是超时的分组

## 拥塞的原因/代价: 场景2

### 现实情况: 重复

- ❖ 分组可能丢失, 由于缓冲器满而被丢弃
- ❖ 发送端最终超时, 发送第2个拷贝, 2个分组都传到



输出比输入少原因: 1) 重传的丢失分组; 2) 没有必要重传的重复分组

### 拥塞的“代价”:

- ❖ 为了达到一个有效输出, 网络需要做更多的工作 (重传)
- ❖ 没有必要的重传, 链路中包括了多个分组的拷贝
  - 是那些没有丢失, 经历的时间比较长 (拥塞状态) 但是超时的分组
  - 降低了的“goodput”

### 4. 拥塞的原因/代价: 场景3

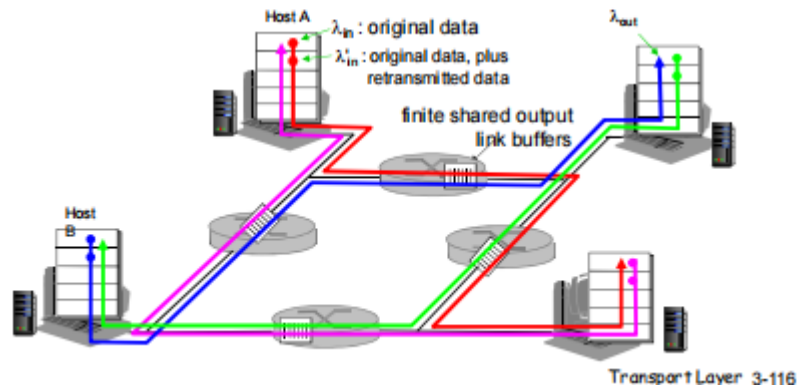


### 拥塞的原因/代价: 场景3

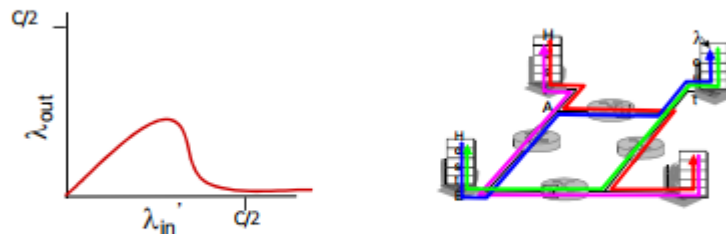
- 4个发送端
- 多重路径
- 超时 / 重传

Q: 当  $\lambda_{in}$   $\lambda'_{in}$  增加时, 会发生什么?

A: 当红色的  $\lambda'_{in}$  增加时, 所有到来的蓝色分组都在最上方的队列中丢弃了, 蓝色吞吐  $\rightarrow 0$



### 拥塞的原因/代价: 场景3



又一个拥塞的代价:

- 当分组丢失时, 任何“关于这个分组的上游传输能力”都被浪费了

5. 拥塞控制方法:

- 端到端拥塞控制:
  - 没有来自网络的显示反馈
  - 端系统根据延迟和丢失事件推断是否有拥塞
  - TCP采用的方法
- 网络辅助的拥塞控制:
  - 路由器提供给端系统以反馈信息
  - 单个bit置位, 显示有拥塞 (SNA, DECbit, TCP/IP ECN, ATM)
  - 显示提供发送端可以采用的速率

6. 拥塞的代价:

- 当分组的到达速率接近链路容量时, 分组经历巨大的排队时延
- 发送方必须执行重传以补偿因为缓存溢出而丢弃的分组
- 发送方在遇到大时延时所进行的不必要重传会引起路由器利用其链路带宽来转发不必要的分组副本
- 当一个分组沿一条路径被丢弃时, 每个上游路由器用于转发该分组到丢弃该分组而使用的传输容量最终被浪费掉了

## 4.TCP拥塞控制

### 1. 机制：

- 端到端的拥塞控制机制：路由器不向主机提供有关拥塞的反馈信息  
路由器的负担较轻；符合网络核心简单的TCP/IP架构原则
- 端系统根据自身得到的信息，判断是否发生拥塞，从而采取动作
- 拥塞控制的几个问题：
  - 如何检测拥塞：轻微拥塞/拥塞
  - 控制策略：
    - 在拥塞发生时如何动作，降低速率
    - 在拥塞缓解时如何动作，增加速率

### 2. 拥塞感知：发送端如何探测到拥塞

- 某个段超时了（丢失了）：拥塞  
超时时间到，某个段的确认没有来  
原因1：网络拥塞概率大  
原因2：出错被丢弃了概率小
- 有关某个段的3次重复ACK：轻微拥塞  
网络这时候还能进行一定程度的传输，拥塞但情况比第一种好

### 3. 速率控制方法：

- 如何控制发送端发送的速率：
  - 维持一个拥塞窗口的值：CongWin, cwnd
  - 发送端限制已发送但是未确认的数控量（的上限）：
$$LastByteSent - LastByteAcked \leq cwnd$$
  - 从而粗略地控制发送方往网络中注入的速率



- cwnd是动态的，是感知到的网络拥塞程度的函数
  - 超时或者3个重复ack, cwnd减小
    - 超时：cwnd降为1MSS，进入SS阶段然后再倍增到cwnd/2，从而进入CA阶段
    - 3个重复的ack：cwnd降为cwnd/2，CA阶段
  - 否则（正常收到ACK，没有上述情况）：cwnd增加
    - SS阶段：加倍增加（每个RTT）
    - CA阶段：线性增加（每个RTT）
- 4. TCP拥塞控制和流量控制的联合动作：发送端控制发送给但是未确认的量同时也不能够超过接收窗口，满足流量控制要求：

$$SendWin = \min(CongWin, RecvWin)$$

##### 5. 拥塞控制策略：

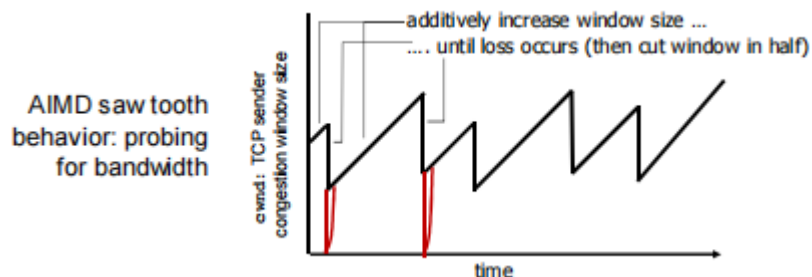
- 慢启动
- AIMD：线性增、乘性减

##### 6. 慢启动 (SS)：

- 连接刚建立：CongWin=1MSS
- 可用带宽可能远大于MSS/RTT，应该尽快加速，到达希望的速率
- 当连接开始时，指数性增加发送速率，直到发生丢失的事件
- 启动初值很低，但是速度很快
- 每一个RTT，CongWin加倍

##### 7. AIMD：

- 乘性增：丢失事件后将CongWin降为1MSS，将CongWin/2作为阈值，进入SS阶段（一直倍增到CongWin/2）
- 加性减：当CongWin大于阈值时，一个RTT如没有发生丢失事件，将CongWin加1MSS
- 当收到3个重复的ACK：CongWin减半；CongWin=Threshold+3,窗口线性增加
- 当超时事件发生时：CongWin被设置成1MSS，进入SS阶段；之后指数增长到CongWin/2，再线性增长



| TCP 发送端拥塞控制           |           |                                                                        |                                   |
|-----------------------|-----------|------------------------------------------------------------------------|-----------------------------------|
| 事件                    | 状态        | TCP 发送端行为                                                              | 解释                                |
| 以前没有收到ACK的data被ACKed  | 慢启动 (SS)  | $CongWin = CongWin + MSS$<br>If ( $CongWin > Threshold$ )<br>状态变成“CA”  | 每一个RTT CongWin 加倍                 |
| 以前没有收到ACK的data被ACKed  | 拥塞避免 (CA) | $CongWin = CongWin + MSS * (MSS / CongWin)$                            | 加性增加, 每一个RTT对 CongWin 加一个 1 MSS   |
| 通过收到3个重复的ACK, 发现丢失的事件 | SS or CA  | $Threshold = CongWin / 2$ ,<br>$CongWin = Threshold + 3$ ,<br>状态变成“CA” | 快速重传, 实现乘性的减. CongWin 没有变成 1 MSS. |
| 超时                    | SS or CA  | $Threshold = CongWin / 2$ ,<br>$CongWin = 1 MSS$ ,<br>状态变成“SS”         | 进入slow start                      |
| 重复的 ACK               | SS or CA  | 对被ACKed的segment, 增加重复ACK的计数                                            | CongWin and Threshold 不变          |

8. TCP吞吐量：忽略SS阶段，因为速度很快，几乎忽略不计

- W：发生丢失事件时的窗口尺寸大小

平均窗口大小： $3/4W$

平均吞吐量：RTT时间吞吐 $3/4W$

$$\text{avg TCP throughput} = \frac{3}{4} \frac{W}{RTT} \text{ bytes/sec}$$

## 5.公平性

---

TCP趋向于在竞争的多条TCP连接之间提供对一段瓶颈链路带宽的平等共享。

在理想化情形下，可以满足TCP公平性。实践中，当多条连接共享一个共同的瓶颈链路时，哪些具有较小RTT的连接能够在链路空闲时更快地抢到可用带宽，因此将比哪些具有较大RTT的连接享用更高的吞吐量。

**参考链接：**

[TCP拥塞控制](#)

[TCP拥塞控制中的算法](#)